

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION WITH OPENCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.* (BTL 6)

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

Question 8 – Motion Detection for Home Security

Problem:

A home security system must detect motion from a camera feed, trigger alerts only for significant movement, and reduce false alarms.

Answer:

Input Acquisition:

Use `cv2.VideoCapture()` to capture live video from a webcam or security camera.

Preprocessing:

Convert each frame into grayscale using `cv2.cvtColor()`. Apply Gaussian Blur using `cv2.GaussianBlur()` to remove small noises.

Motion Detection:

Compare the current frame with the previous frame using frame differencing. Apply thresholding using `cv2.threshold()` to identify areas where significant changes occur.

Noise Removal:

Use morphological operations such as erosion and dilation to remove small unwanted regions. A minimum contour area is selected so that very small movements do not trigger an alarm.

Motion Identification:

Use `cv2.findContours()` to detect moving objects. If the detected contour area is greater than the predefined threshold, motion is considered significant.

Alert Generation:

When significant motion is detected, the system can generate an alarm or notification and display the detected region using a bounding box.

Pipeline:

Camera → Grayscale → Gaussian Blur → Frame Difference → Threshold → Morphological Filtering → Contour Detection → Motion Alert

Justification:

This method is suitable for home security because it is simple and computationally efficient. Blurring reduces noise, thresholding detects changes, and contour-area filtering reduces false alarms caused by small movements. It can therefore operate in real time with limited processing requirements.

Question 9 – Document Scanner Application**Problem:**

A mobile application must capture document images, enhance readability, and produce clean scanned images.

Answer:**Input Acquisition:**

Capture the document using a mobile camera or webcam using OpenCV.

Preprocessing:

Convert the captured image into grayscale using `cv2.cvtColor()`. Apply Gaussian Blur to reduce noise.

Edge Detection:

Use the Canny edge detection algorithm with `cv2.Canny()` to identify the boundaries of the document.

Contour Detection:

Use `cv2.findContours()` to detect the document boundary. Select the largest four-sided contour as the document region.

Perspective Correction:

Apply a perspective transformation using `cv2.getPerspectiveTransform()` and `cv2.warpPerspective()` to convert the tilted document into a rectangular view.

Image Enhancement:

Apply adaptive thresholding or binary thresholding to improve text visibility. Contrast enhancement can also be applied when required.

Output:

Display or save the final corrected document as a clean scanned image.

Pipeline:

Camera → Grayscale → Noise Removal → Canny Edge Detection → Contour Detection → Perspective Transformation → Thresholding → Scanned Output

Justification:

Perspective correction removes camera-angle distortion, while grayscale and thresholding improve readability. Noise removal makes edge detection more reliable. The complete pipeline provides a clean document image suitable for digital storage and sharing.

Question 15 – Real-Time Edge Detection System

Problem:

The system must capture video, detect edges in real time, and display the results efficiently.

Answer:

Input Acquisition:

Capture live video using `cv2.VideoCapture()`.

Preprocessing:

Convert each frame into grayscale using `cv2.cvtColor()`. Apply Gaussian Blur using `cv2.GaussianBlur()` to reduce noise.

Edge Detection:

Use the Canny edge detection algorithm with `cv2.Canny()`. It detects important boundaries and edges present in the image.

Display:

Display the original frame and edge-detected frame using `cv2.imshow()`.

Optimization:

To achieve real-time performance, the input resolution can be reduced. Processing can also be performed on grayscale images instead of full-color images. Unnecessary operations should be avoided.

Pipeline:

Camera → Frame Capture → Grayscale Conversion → Gaussian Blur → Canny Edge Detection → Display

Justification:

Canny edge detection is effective for identifying object boundaries. Grayscale conversion and image resizing reduce computational requirements. These optimizations allow the system to process video continuously with minimum delay.

Code of Conduct:

I V G ABBIRAMY - 192524097 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V G ABBIRAMY

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization

Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

		tools			
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided